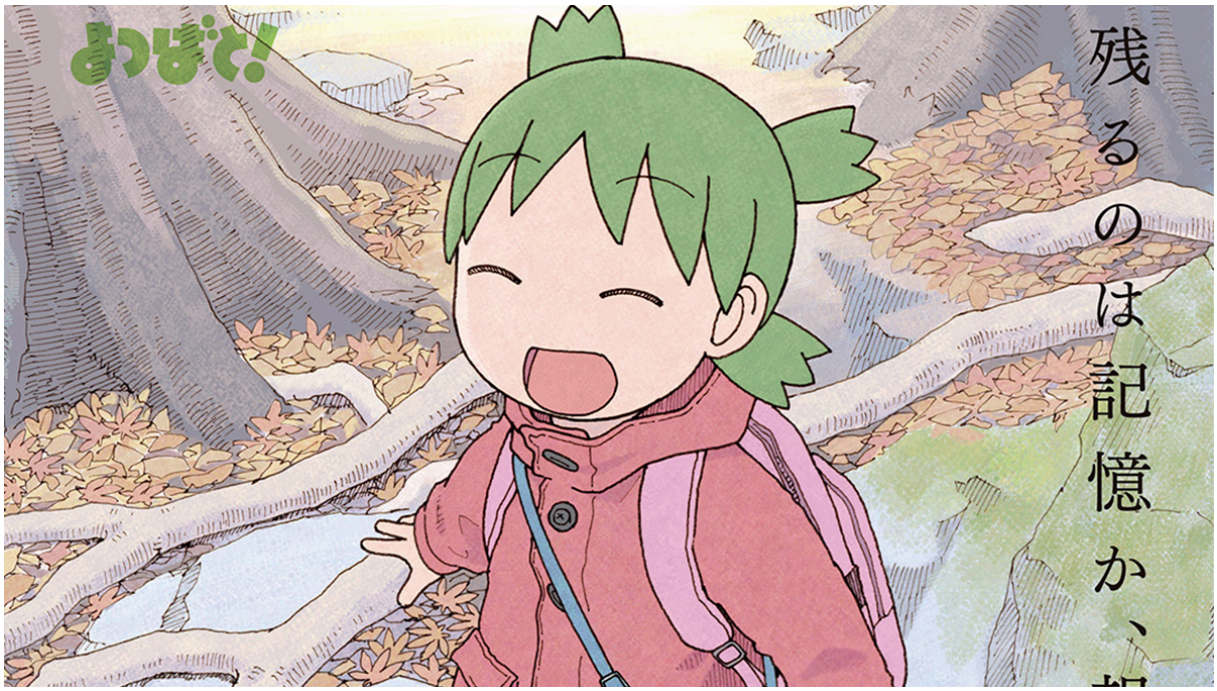
CMSC 216 Notes

Introduction to Computer Systems

Miagao



Yotsuba from Yotsuba&!

Contents

1 Course Intro 5
2 C Language I 6
- C versus Java 6
- C Program Example 6
- Compiling a C program 6
- Compilation Stages 7
- Types in C 7
- Numerical Literals 8
- Variable Declaration 8
- Arithmetic Operators 8
- Boolean Type and Operators 9
- Precedence and Associativity 9
- Assignment Statements 9
- Output using `printf()` 10
- Input using `scanf()` 10
- C Control Statements 11
- Additional Formatting Options 11
- C Functions 11

3 C Pointers 13
- The `sizeof` operator 13
- Pointers 13
- Dereferencing a Pointer 14
- Pointers as Parameters 14
- Garbage Pointers 15
- Pointers to Local Variables 15
- `NULL` Pointer 15
- The `const` modifier 15

4 Arrays 17
- Arrays in C (static) 17
- Array Initialization 17
- Symbolic Constants 17
- Array Parameters 18

5 C Pointers II 19
- Incrementing Pointers 19
- Pointer Arithmetic 19
- Generic Pointer 20
- Pointers to Pointers 20
- Arrays versus Pointers 20
- Arrays of Pointers 21

6 Strings 22
- Strings in C 22
- Initializing Strings 22
- Defining String Variables 23

`scanf()` 23
Basic String Library Functions 23
String Literals 25
ctype.h 25
7 Command Line Arguments and 2-Dimensional Arrays 26
Command Line Arguments 26
Accessing Command Line Arguments 26
2-Dimensional Arrays 26
8 Structures 28
The `typedef` Keyword 28
Struct Definitions 28
Struct Initialization 29
Struct Assignment and Comparison 29
Structs and Functions 29
The -> Operator 30
Struct Storage 30
Anonymous Structs 31
Unions 31
9 IO 33
Exit Codes 33
Errors 33
Standard IO 33
Opening/closing Files 34
Line-oriented IO 34
Streams 35
`sscanf()` 35
`sprintf()` and `snprintf()` 36
Reading data from strings 36
Working with long lines 36
Shell Programs 36
10 Dynamic Memory Allocation 38
Memory Organization 38
Dynamically Allocated Memory 38
Memory Management Functions 38
Pointer Aliases 39
Common Errors 39
11 Dynamic Memory Allocation 40
Linked Lists 40
Operations 40
Dummy Nodes 41
Doubly linked Lists 41
Reallocation 41
Operating on blocks of memory 41
12 Lab 1 and 2 42

Wildcard and Path symbols 42
Linux Shell Commands 42
13 Lab 4 44
`diff` and I/O redirection 44
Valgrind 44
gdb 45
splint 45
assert 46
Compiler Options 46
14 Lab 5 47
`emacs` 47
15 Lab 6 48
Comma Operator 48
C Memory Organization 48
Identifier Scope 48
Storage 49
Identifier Linkage 49
Enumerated Types 49
Lvalues and Rvalues 50
Typecasting 50
16 Lab 8 52
Standard Library (continued) 52
String Library (continued) 52
Command Line Arguments (continued) 53
2-dimensional Arrays (continued) 53
17 LAB 54
File Parsing 54
Preprocessing 54
Conditional Compilation 54
File Inclusion 55

1 Course Intro

C language is a high-level language close to assembly.
- Exposes how data is organized in a linear address space.
- **Pointers** demonstrate how variables represent locations in memory.
- Shows system calls provided by the OS.

Assembly language shows what hardware can do:
- Instructions processors can execute.
- How function calls work.
- How input/output devices are accessed.

UNIX operating system
- Shows how threads, processes, pipes, etc are building blocks for servers and high-performance applications.
- We will use a variant of UNIX called **Linux**.

2 C Language I

For this course, we will be using C99.

- C is not object oriented, so instead of the term *method*, we use **function**.
- To stop a C program, use ctrl + c.

C versus Java

- C is more performant because it trusts the programmer to manage details such as array bounds, memory leaks, and error checking.
  - Java's garbage collection and constraints hurt performance.
- C is the next level after assembly, thus it's used for writing operating systems such as Unix.
- C compiles to machine code which the processor can directly execute. Java requires an intermediate step of converting to bytecode.
- As mentioned prior, C lacks explicit object-oriented features.

C Program Example

```c
#include <studio.h>
int main() {
  printf("Fear the turtle\n");
  return 0;
}
```

- `#include` allows the compiler to include other files (generally .h **header files**) for their function prototypes and type/variable declarations.
  - Standard system files are delimited by **<>** while user-written files are delimited by **“”**.
- The `main()` function is where all C programs begin execution.
  - Return values from this function indicate the end of a program.
  - By convention, an exit code of 0 means that everything went well.

Compiling a C program

C programs must be compiled to be executed.

- The **gcc** program is will be used to build our programs.
  - `gcc SOURCE_FILES`
- The common flags for gcc are as follows:
  - `-g` enables debugging.
  - `-Wall` warns about potential problems.
  - `-o FILENAME` places the executable in the specified file.
  - `-c` compiles to object file (without link).
  - `-lm` compiles with the C math module (if imported).

The following are examples of compiling single files and multiple files at once.

```shell
gcc file.c
gcc -g -Wall -o prog1 prog1.c tools.c
```

To run executables in UNIX, type the name of the executable. In some cases, the executable's name should be preceded by "./".

Compilation Stages

- **Preprocessor**
  - *Does text-processing of source code.*
  - Used to provide declarations shared by different source code.
  - Directives begin with the pound sign, #, but do not end with a semicolon.
- **Translation**
  - *An object file **(.o)** is created.*
  - Ensures individual files are consistent within themselves.
- **Linking**
  - *Joins one or more compiled object files together.*
  - *Creates an executable file.* Note that by UNIX convention, executable files lack a filename extension

Types in C

- char, short, int, long int.
  - These can be signed or unsigned.
- float, double
- * is a pointer to a variable.
- [] represents arrays of variables.
- union {} represents variables sharing the same space.
- struct {} represents the concatenation of variables.
- enum {} represents named integer values.

Type name	Minimum size (bytes)	Size on *grace* (bytes)
char	1	1
short	2	2
int	2	4
long	4	8
float	4	4
double	8	8
long double	10	16

Numerical Literals

Suffixes can be used to specify a numerical literal's type.
- "L" can be used to specify "long."
- "f" can be used to specify "float."
  - By default, a decimal is a double.

Prefixes can be used to specify bases other than base 10.
- "0" represents octal.
- "0x" represents hexadecimal.

Variable Declaration

Variables may be declared as follows.

typename var1, var2, ...;

Variables may also be initialized during declaration.

```c
int x = 6;
```

Even if an initial value is not given, declaration always reserves space for the variable in memory. In the example above, 4 bytes will be reserved even if the initial value of 6 was not given.
- However, it is recommended to give an initial value, as C will not stop you from using an uninitialized variable. In fact, it will use whatever data is in the reserved memory space.

Arithmetic Operators

The basic arithmetic operators in C are analogous to Java.
- + add
- - subtract
- / divide
- % remainder
- * multiplication

Compound assignment operators are basic arithmetic operators followed by an equals sign.
- +=, -=, /=, %=, *=
- ++ increment
- -- decrement
- **(typename)** casting

> **Note**
>
> Integer division occurs only if both operands are integers. Otherwise, floating point division occurs.

Boolean Type and Operators

There is no boolean type in C. Integers are used instead.
- 0 represents **false**.
- Any other integer represents **true**. By convention, this is 1.

Similar to arithmetic operators, the boolean operators are analogous to Java.
- Relational operators: <, >, <=, >=
- Equality operators: ==, !=
- Logical operators: &&, ||, !

Because integers are used instead of a true boolean type, these either evaluate to **0 (false)** or **1 (true)**

C
```c
int i;
i = (!3) == (4 < 2);
i = (!2) || (5 && i);
```

Java
```java
boolean i;
i = (!true) == (4 < 2);
i = (!true) || (5 && i);
```

Precedence and Associativity

some of these fellas go first. Lowk PEMDAS.

Assignment Statements

The **assignment operator** in C is the equals symbol, =.
- It is right associative. A value is returned from the operator to the variable on the left side of the assignment operator.
- *Note, this means that variable assignment is an expression. Consequently, you can write any expressions as individual statements.*

Generally, expression statements are most useful with the assignment operators.

C
```c
int x, y;
y = x = 123; //x is assigned to 123, then y is assigned to 123 transitively through x.
y = 5 + (x = 3); //x is assigned to 3 which yields 3, then y is assigned to 5 + 3 = 8.
```

Output using `printf()`

The `printf()` function is formally declared in stdio.h. It can print strings, which may be formatted with a variable number of expression statements.

---

printf(“formatstring”, expression1, expression2, expression3, …**)**;

The expressions are inserted into the format string using format specifiers. These correspond to each passed-in expression, and decide how each value will be interpreted.

- %d prints an integer in decimal.
- %ld prints a long integer.
- %u prints an unsigned integer.
- %f prints a floating point value.
- %c prints a character.
- %s prints a character string.
- %x prints in hexadecimal.
  - %X makes the letters capital.
- %e prints in exponential form (e notation).
- %% prints the percent symbol.

Like Java, **escape sequences** can be used in strings to specify things that would be otherwise inconvenient to type out.

- \n creates a new line.
- \t creates a horizontal tab.

Input using `scanf()`

The `scanf()` function is formally decalred in stdio.h.

---

scanf(“formatstring”, **&**variable1, **&**variable2, **&**variable3, …**)**;

The format string specifies the inputs being read using format specifiers and inserts them into each passed-in variable.

- Format specifiers should match the type of their corresponding variables. If this is not the case, the function will cease.
  - *Most* specifiers skip leading whitespace.
- Whitespace in the format string simply skips any whitespace input at that point.
- Other characters just tell the function to read the corresponding characters in the input stream.

The `scanf()` function **returns the number of items assigned to variables or the EOF (End of File Marker).**

> **Note**
>
> The ampersand symbols associated with each variable means that the address of each variable is being retrieved.

> **Caution**
>
> Avoid adding whitespace at the end of the format string of a `scanf()` function.

C Control Statements

Just like Java, C has **if**/**else**/**else if**, **for**, **while**, and **switch** statements.

The flow of loops can be controlled using the following keywords:
- **break** immediately ends a loop.
- **continue** skips the remainder of the loop and advances to the next iteration.

Compound statements are delimited by {}, and can go anywhere a single statement can. Variables defined within are limited to the scope of the compound statement.

Additional Formatting Options

We can specify field width, precision, etc alongside format specifiers.
- **-** left-aligns the output within the field width.
- **0** pads the output with leading zeros when possible.
- **+** displays a plus sign for positive signed numbers.
- An integer specifies the width (minimum characters).
- An period followed by an integer specifies the precision.
  - Floating-point: number of digits
  - Integer: minimum digits
  - String: maximum characters

C Functions

Functions in C follow the following syntax:

---

returnType functionName(parameters) {/* statements */}

They are called as follows:

---

functionName(arguments);

Function prototypes provide information about a function's return type and parameter types, but do not define it.
- The compiler can use them to verify the correctness of function calls.

- Thus, they allow functions to be defined after the main function in a C program.

---

returnType functionName(parameterTypes);

> **Important**
>
> Unlike Java, functions **cannot be overloaded.**

> **Caution**
>
> Previously, variables in compound statements could only be declared at the top. In C99, mixed declarations are now allowed. This primarily affected for-loops.

3 C Pointers

The `sizeof` operator

The `sizeof` operator is *unary*, and evaluates to the bytes needed to store the operand.

- The operand can be either an expression or a type name.

> **Note**
>
> The expression is not evaluated.

- Return type is long.
- For example:

```c
int i = 67;
printf("%ld\n", sizeof(i)); //Prints 4 (on grace)
printf("%ld\n", sizeof(unsigned char)); //Prints 1
printf("%ld\n", sizeof(++i)); //Prints 4 (on grace)
printf("%ld\n", i); //Prints 67, the line 4 expression isn't evaluated
```

Pointers

A **pointers** is a memory address or reference to a variable. They have two attributes:

- **Address:** Address of a byte of memory.
- **Type:** Type of the data starting at the address.
- A **pointer variable** is a variable that holds a pointer value.
  - Many times, they are referred to as just *pointers*.

Variables are denoted as pointers with an asterix (*) before it:

```c
int *p; //Creates a variable ip, whose type is "pointer to int"
```

- Due to how they're denoted, the following are also equivalent:
  - `int*p`
  - `int * p`
  - `int* p`

An ampersand (&) before an entity *denotes pointer to it* (specifically, the address of its first byte).

The *address of an int variable* can be the value of an *int pointer variable.*

- We can access the entity a pointer variable points to.
- The type attribute of a pointer is used by the compiler when generating object code, but the attribute itself is not stored in object code.

```c
int i = 6;
int *p;
```

```c
p = &i;
printf("%d %d\n", *p, *(&i));
```

While addresses look like integers, they are a different type. For instance, the following does not compile:

```c
int x = 20;
int *m = x;
```

> **Note**
>
> The format specifier for addresses is `%p`, although it requires a null pointer variable for use in `printf()`. Thus, casting using `(void *)` can be used.

> **Note**
>
> The size of a pointer depends on the system. Pointers in a 32-bit system are 4 bytes and pointers in a 64-bit system are 8 bytes.

Dereferencing a Pointer

An asterix (*) before a pointer *dereferences the pointer*.

By dereferencing a pointer variable, we can access the value present at the address being pointed to.

```c
int i = 6, j, *p;
p = &i;
j = *p; //Dereferences p, j is set to 6
*p = 4; //Points to the address of 4
```

Pointers as Parameters

> **Important**
>
> Everything in C is passed by value.

Below is a good example of a function that needs pointers as parameters

```c
void swap(int *a, int *b) {
  int temp = *a;
  *a = *b;
  *b = temp;
}
```

Because things are passed by value, pointers are needed to change the values of the variables provided in the function call.

Garbage Pointers

When a pointer is defined, it points to the address associated with the memory location allocated for the pointer variable. If pointed to an address where no value has been initialized:

- A **segmentation fault** (**segfault**), accessing a memory location that you don't have permission to access.
- A **bus error**, violating the computer's memory alignment policies.
- A **silent failure**, where everything appears to run correctly, but the value being accessed may change later on.

Pointers to Local Variables

> **Caution**
>
> Do not return pointers to local variables. Once the function call has finished, the memory used by the variable is not supposed to be accessed.

NULL Pointer

A **NULL pointer** points to the address 0, where nothing is allowed to be accessed.

- This is defined in stddef.h, which is included by many header files.

```c
#define NULL ((void*)0)
```

- **NULL** is analogous to **null** in Java.
  - Dereferencing a NULL pointer usually results in a segfault, as you are generally not supposed to access the address 0.
- NULL can be assigned to any kind of pointer variable, so checking whether or not a pointer is NULL prior to dereferencing is safest.

The `const` modifier

The `const` modifier indicates that a variable cannot be changed. This is enforced by the compiler. The order of the type specifier and the modifier does not matter.

```c
int const k = 4;
const int m = 5;
k++; //Error
m++; //Error
```

The order of the type specifier and the modifier *does matter when using pointers*:

```c
int i = 4, j = 5;
const int *p = &i; //Pointer variable to constant int
int * const q = &j; //Constant pointer variable to int

p = &j; //WORKS
*p += 5; //ERROR
q = &i; //ERROR
*q += 23; //WORKS
```

- If a pointer variable points to a constant value, then it is that value that cannot be changed.
- If a constant pointer variable points to a value, then it is the pointer value that cannot be changed.

4 Arrays

Arrays in C (static)

Like Java arrays, all elements in an **array** are of the same type and indexed using the [] operator.

- In C, elements of an array are next to each other in memory as opposed to being an object like in Java.
- Additionally, the user has to keep track of the size, as there is no length property.
- The syntax between them is slightly different too.

```c
//array of 5 ints
int a[5];
```

```java
//array of 5 ints
int[] a = new int[5]
```

Arrays can be viewed as a pointer to the first element of the array.

- *The name itself is not a pointer.* Rather, the compiler converts it to a pointer when needed.
- The (constant) size of an array must be known at compile time.

> **Caution**
>
> An assignment operator cannot be used with two array operands.

However, using == and != does work. However, this does *pointer comparison*.

Array Initialization

When initializing arrays, you can list the elements separated by commas and in-between braces:

```c
int arr[5] = {1, 2, 3, 4, 5};
```

- Because assignment differs to initialization, the notation above does not work if the array has been previously declared.
- 0s pad the array if the initializer is not complete.
- A size does not need to be provided for an initialized array, as the size is *implicit*.
- Initializers with excessive elements cause compiler errors.
- Initializing with no elements in-between braces causes a compiler error.

Symbolic Constants

Symbolic constants can be created using the #define preprocessor directive:

```
#define name value
```

All occurences of *name* in the program will be replaced with *value*.

- This is useful for defining constants for important values such as array sizes to improve code maintainability.

```c
#include <stdio.h>
#define ARRAY_LENGTH 3

int main() {
  int i, a[ARRAY_LENGTH] = {10, 20, 30};
  printf("[%d", a[0]);
  for (i = 1; i < ARRAY_LENGTH; i++) {
    printf(", %d", a[i]);
  }
  printf("]\n");

  return 0;
}
```

Array Parameters

Recall that arrays are pointers to the first element of the array.

- The parameter declaration int a[] is equivalent to int *a.
- Thus, modifications done to an array parameter are done to the passed-in array.
- Consequently, the defined array parameter size does not matter. *Only the type matters.*

> **Note**
>
> Passing in the array parameter's size as a different parameter is common practice.

5 C Pointers II

Incrementing Pointers

Pointers can be incremented or decremented like integer type variables. This “moves” one element at a time.

- As a result, the number of bytes of memory moved depends on the pointer’s type.
- This can be seen with arrays because their elements are in contiguous memory addresses.

> **Important**
>
> Incrementing pointers only make sense with arrays, which are pointers to their first element.

> **Note**
>
> The postfix operator takes precedent over the dereferencing operator (*) and the prefix operator (++var).

- The dereferencing operator (*) and prefix operator have the same precedence level, right-to-left associativity.

```c
int o = 10;
int *p = &o;
++*p; //Increments the value at the location where p points (o = 11), then evaluates incremented value (11)
*p++; //Evaluates to the value at the location where p points (11), then increments p.
p--;
(*p)++ //Evaluates to the value at the location where p points (11), the increments that value (o = 12).
```

Pointer Arithmetic

By adding an integer n to a pointer, we can get the the address of the nth element past the element where the pointer currently points:

- The only valid forms of pointer arithmetic are:
  - pointer - pointer
  - pointer $\pm$ integer
- When two pointers are pointing to the same array, we can determine how far apart they are by finding their difference

Generic Pointer

Generic pointers are pointer variables defined as `void*`.

- The value of any pointer variable can be assigned to a generic pointer.
- Casting to `void*` is not needed to assign to a generic pointer.

```c
int x = 20;
void *p = &x; //This assignment works
```

Generic pointers *cannot be dereferenced*. This is why `NULL` pointers cannot be dereferenced.

- When a pointer is dereferenced, the number of bytes associated with the entity being pointed to must be known.

> **Important**
>
> The address in the generic pointer must be casted prior to dereferencing.

```c
printf("%d\n", *(int *)p);
```

Pointers to Pointers

The address of a pointer variable can be obtained.

```c
int i = 4, j = 6;
int *p = &i, *q = &j;
int **r = &p; //Double pointer, points to the address of p
printf("%d\n", **r); //Double dereferences to access the value of i
*r = &j; //Dereferences to change the value of p
printf("%d = %d\n", *p, *q); //Both should print the value of j
```

- This is useful when using pointer variables as parameters

Arrays versus Pointers

```c
int nums[3];
```

- Declares an array, allocates 3 ints worth of space, and points the name `nums` to the beginning of this space.
- By itself, `nums` is treated as a **constant pointer** that points to the beginning of the array.
- Recall that `nums` *cannot* be changed to point to another space.

```c
int *nump
```

- Declares a pointer, doesn't allocate anything more than the space needed to store an address, then the name `numps` points to this space.
- `nump` can be changed and assigned to.

Arrays of Pointers

Arrays of pointers are similar to arrays in Java.

```c
int *nums[3];
```

- An array of 3 pointers to int.

Arrays of pointers are useful when dealing with structures of different sizes.

- Allows pointers to be sorted with moving around lots of memory.

The argv array for a command line is implemented as an array of char pointers.

- Analogous to the args array of Strings in Java.

6 Strings

Strings in C

There is no String type in C. Internally, they are defined as arrays.

- In C, a string is defined as a sequence of characters that si followed by a byte of value 0. This is known as a **null-terminating string**.
  - This 0 byte is often called the **zero byte**, **null byte**, or **NUL**.
  - Represented by the character literal **\0**.
  - According to the ascii table entry for NULL:
    - `man ascii`
    - The \0 is a 0, and therefore is false in C.
    - Strings are *arrays of characters with a 0 at the end*.

Now, because arrays are contiguous in memory, and chars are one byte each, arrays of chars can be used to contain strings.

> **Note**
>
> All strings in C are character arrays, but not all character arrays are strings. This is because strings *require a null byte at the end*.

Initializing Strings

Because strings are closely related to character arrays, we can initialize character arrays to strings *using both string literals and standard array initializers*.

- For instance, the following strings are equivalent.

```c
char str1[] = "Hello";
char str2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

- Implicitly, the sizes of these character arrays would be 6.

Because strings are character arrays, they cannot be initialized using assignments.

- Instead, we can use the function **strepy** to copy a string.

Additionally, empty spaces in strings are padded by null bytes:

```c
char a[] = "hello"; //{'h','e','l','l','o','\0'}
char b[10] = "Maryland"; //{'M','a','r','y','l','a','n', 'd','\0',`\0`}
char c[1024] = "" //{'\0','\0', ...} (1024 null bytes)
```

We can also use `const char *str` for strings. The const modifier is to enforce pointer safety because we can not reassign `str`.

Defining String Variables

When defining the size of a string variable, the actual number of characters that can be stored is one less than the size.

- For this course, we will use symbolic constants to define the size, then add 1 when declaring strings to account for the null byte.

```c
#define MAX_LENGTH 80

char str[MAX_LENGTH + 1];
```

`scanf()`

Recall that the conversion specifier to read a string is %s.

- `scanf()` with %s skips spaces until it finds a non-space character, then keeps reading characters until it reaches a space.
  - Reading lines with spaces between numerous groups of characters requires another function.

> **Important**
>
> The array must be large enough to read the data, as `scanf()` will not check if the array's size is valid. Thus, it may write characters to memory spaces that it should not be overwriting.

- Reading n characters requires an array of size $n + 1$ due to the null byte.
  - For instance, 1%s can read one character, but must be written to a char array of at least size 2.
- Because strings are character arrays, associated variables hold the address. Thus, an ampersand is not needed when used as an argument to `scanf()`.

> **Note**
>
> After reading a string, `scanf()` adds a null character to the end.

Basic String Library Functions

C has standard functions for strings in the string header file. Like stdio.h, a linking flag is not required when compiling the program.

```c
#include <string.h>
```

The **size_t** type is defined in C as an unsigned integer (unsigned long on grace). It is capable of representing the size of the largest possible object in the computer.

strlen is defined as follows:

```c
size_t strlen(const char *s);
```

- Returns the length of the given string, *disregarding the null byte*.
- The sizeof operator should not be used to find the length of a string.
  - If passed to other functions, sizeof will return the system's pointer size.

Below is a possible implementation of strlen:

```c
size_t strlen(const char *s) {
  size_t i;

  for (i = 0; s[i]; i++);

  return i;
}
```

- The function stops when the null byte is read, so the length of an uninitialized string will be 0.

strcmp is defined as follows:

```c
int strcmp(const char *s1, const char *s2);
```

- Analogous to `String::compareTo` in Java.
  - Returns a negative number if `s1` < `s2`.
  - Returns a positive number if `s1` > `s2`.
  - Returns 0 if `s1` = `s2`.
  - Comparisons are done per character and alphabetically.

Below is a possible implementation of strcmp:

```c
int strcmp(const char *s1, const char *s2) {
  int i;

  for (i = 0; s1[i] && s2[i]; i++)
    if (s1[i] != s2[i])
      break;

  return s1[i] - s2[i];
}
```

strcpy is defined as follows:

```c
void strcpy(char *dest, const char *src);
```

- Copies the string in `src` to `dest`.
- The strings may not overlap.

> **Caution**
>
> The function does not know if `dest` is large enough to hold the string from `src`.

Below is a possible implementation of strcopy:

```c
void strcpy(char *dest, const char *src) {
  int i = 0;

  while (src[i]) {
    dest[i] = src[i];
    i++;
  }
  dest[i] = '\0';
}
```

String Literals

String literals are denoted by the double quotes. They are initially part of read-only text.

- String literals containing one character are different from character constants. Specifically, the literal would take up more space due to the null byte.

Additionally, *character arrays are different from character pointers.*

```c
char str1[] = "BANG"; //String variable, mutable
char *str2 = "BANG"; //Pointer to string literal, immutable
```

- The pointer should be decalred as `const char *str2`.
- Either choice works as an argument to a function expecting a character array or character pointer.
- Two pointers to string literals can share the same string.

ctype.h

The header file **ctype.h** provides functions for character types.

```c
int isalphanum(int); //Checks if given char is alphanumeric
int isalpha(int); //Checks if given char is a letter
int isdigit(int); //Checks if given char is a decimal digit
int isblank(int); //Checks if given char is a space or tab
int isspace(int); //Checks if given char is whitespace
int islower(int); //Checks if given char is a lowercase letter
int tolower(int); //Returns lowercase version of given char
int isupper(int); //Checks if given char is an uppercase letter
int toupper(int); //Returns uppercase version of given char
```

7 Command Line Arguments and 2-Dimensional Arrays

Command Line Arguments

Executing commands such as “ls -l” or “emacs puzzles.c”, the various components of the command line are passed as arguments to the program.

- These can be accessed using string parameters, akin to `String args[]` in Java.

Accessing Command Line Arguments

The program below will print out each argument in the command line on separate lines.

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
  int i = 0;

  for (i = 0; i < argc; i++) {
    printf("Arg #%d: %s\n", i, argv[i])
  }
  return 0;
}
```

Note that we can also declare `char *argv[]` as `char **argv`.

- When commands are passed into argv, the null byte is automatically added to the end.
- Below is an example of the contents `char *argv[]`.

```c
char *argv[] = {"prog", "1", "two", "THREE", "-4", NULL};
```

- `argv[0]` is generally the name of the executable.
- The **atoi** function from stdlib.h is commonly used to convert numeric command line arguments from strings to integers.

2-Dimensional Arrays

2-dimensional arrays are arrays of arrays.

- When passing 2-dimensional arrays to a function, the size of the second dimension should be specified, but the size of the first dimension is optional.

```c
#define MAX_LENGTH 80
#define MAX_LIST_LENGTH 50

```

```
4    char list1[][MAX_LENGTH + 1];
5    char list2[MAX_LIST_LENGTH][MAX_LENGTH + 1];
```

> **Important**
>
> The compiler needs to know the length of the inner arrays because it needs to know the size to allocate to the outer array's contents.

- 2-dimensional rows *are stored in row-major order*, making them similar to 1-dimensional arrays in memory.
  - A two-dimensional array of characters can also be seen as a one-dimensional array of strings.

8 Structures

For those who didn't know, struct is slang for structure.

The `typedef` Keyword

The keyword `typedef` can be used to define existing types under new names.

```c
typedef double Dollars;
Dollars x = 2.5, y = 1.5;
```

- Internally, these are still doubles.

> **Important**
>
> The naming convention for new type names is to capitalize the first letter of the name.

These are different from using symbolic constants to specify types.
- Symbolic constants replace all specified instances during preprocessing.
- **`typedef` maps all instances to the specified type during compilation.**

Notably, `typedef` can represent pointer types and array types.

```c
#define INT_PTR_DEFINE int*
typedef int* INT_PTR_TYPEDEF

INT_PTR_DEFINE p1, p2; //Symbolic constants are macros, only 1 pointer
INT_PTR_TYPEDEF p3, p4; //Both are pointers, typedef creates a type
```

> **Note**
>
> `typedef` is also usually used with internal types such as `uint8_t` for consistency across different systems.

Struct Definitions

Structs (structures) can hold multiple items of different times. Instead of using indices, different items are referred to by their field names (variable names).
- They are similar to classes in Java, albeit without methods.
- They are denoted by the keyword `struct`.
- Fields are accessed using a dot (.), similar to accessing public instance variables in Java.
  - This operator is left associative.

```c
struct pixel {
  int x, y;
  char color;
};
```

When creating variables using the struct type, we have to use the `struct` keyword and the name of the struct:

```c
struct pixel p1;
```

Struct Initialization

Initializing structs is similar to initializing arrays. Each item in the initializer corresponds to each field in the struct (by order of definition in the struct).

```c
Pixel p1 = {2, 4, 'b'};
Pixel p1 = {2};
```

- Any uninitialized fields will be 0, similar to arrays.
- Additionally, we cannot use the struct initializer during assignment.

Struct Assignment and Comparison

Struct declarations using happen outside of functions and often in header files when used across multiple files.

Struct variables can be assigned to other struct variables of the same struct, copying the bytes for corresponding fields (**shallow copy**).

- Thus, array and pointer fields will be copied, but both struct variables will be referring to the same memory space for their fields.

However, comparison between struct variables of the same struct does not work.

> **Note**
>
> For struct pointers, assigning different struct types is legal through pointer casting.

Structs and Functions

> **Important**
>
> Struct arguments are passed by value. Structs can also be returned from functions.

```c
void print_pixel(Pixel p) {
  printf("x: %d, y: %d, color: %c\n", p.x, p.y, p.color);
}
```

```
4   Pixel increase_x_value(Pixel p, int delta) {
5     Pixel new_pixel = p;
6     new_pixel.x += delta;
7
8     return new_pixel;
9   }
```

Using struct pointers can be more efficient, especially when passing large structs are arguments.

The -> Operator

Given a pointer to a struct, the `->` operator can be used to reference the struct's fields.

- The dot operator may only be used after dereferencing a struct pointer. Due to precedence, parentheses are needed for this operation.

```c
1   Pixel increase_values(Pixel *p, int delta) {
2     p->x += delta;
3     (*p).y += delta; //Does the same thing, but for the y field
4
5     return new_pixel;
6   }
```

> **Note**
>
> By convention, the arrow operator is preferred over the dot operator for struct pointers. Additionally, whitespace is generally not used between the arrow and its operands.

Struct Storage

In grace, the size of structs is always divisible largest field's size, and is generally the minimum needed to store everything.

- Thus, structs often use more memory space than the sum of their fields' storage. This extra space can be seen as "padding."

```c
1   struct one {
2     double b;
3     int a;
4   } s1;
5   printf("%d\n", sizeof(s1)); //Prints 16, but only 12 bytes needed
```

- We can say:

$$0 \leq \text{padding} < \text{largest type size}$$

```c
struct two {
  int b;
  char a;
  short c;
} s2;
printf("%d\n", sizeof(s2)); //Prints 8, but only 7 bytes needed
```

The reason for this is memory alignment issues. For instance, integers must begin at 4-byte boundaries.

Anonymous Structs

```c
struct {
  int id_number;
  char last_name[10];
  char first_name[10];
  double salary;
} emp1, emp2;
```

Notice how there is no tag. This is an **anonymous struct**.
- Specifically, this declares two variables: emp1 and emp2.

Omitting the tag means that a *unique struct type* is created for the listed variables.
- This holds even if other structs with the same fields are created.
- These structs cannot be passed as function arguments.

> **Important**
>
> To improve syntax, `typedef` can be used with structure definitions. *Anonymous structures are often used with typedef as the tag's functionality is largely replaced by the new type definition.*

Unions

Unions are defined similarly to structs.
- However, all fields share the same memory space, so they are only as large as the largest field.
- Consequently, only one field may be valid at a time.
  - The unused fields will contain garbage values.
- Accessing fields also requires the dot operator (.).

```c
typedef union {
  int i;
  double d;
} Number;
```

A more effective way of using unions is to pair them with enums and structs. Inside a struct, an enum can be used to keep track of the type stored within an accompanying union.

```c
typedef struct {
  enum {INT, DOUBLE} type;
  union {
    int i;
    double d;
  } value;
} Number;
```

9 IO

Exit Codes

Exit codes are values associated with program terminations. Many important codes are stored in header files.

- stdlib.h - EXIT_SUCCESS (0), EXIT_FAILURE (1)
- sysexits.h - EX_OSERR (71), EX_USAGE (64)

Errors

perror prints a message to standard error describing the last error encountered during a library function or system call.

- If a string is provided, it will be printed before the default error description.
- perror is based on the **errno macro** in errno.h which returns an associated error code.

> **Note**
>
> The errno macro is a function call that returns a pointer to a location that is only initialized when there are errors.

Using errno as an argument to the **strerror** function, a string library function, we can generate a string for a specific errno.

Standard IO

stdio.h contains prototypes and constants for IO routines. IO is mainly **stream-based** and **buffered** for efficiency.

- The function `fflush()` can be used to flush bushers immediately.

There are two kinds of files in standard IO: **text files** and **binary files**. Accordingly, there are two different stream types associated with each IO file type.

- **Text streams:**
  - Streams composed of text lines, each terminated with a newline character.
  - Library functions can deal with text streams in many ways.
    - Character-oriented IO
    - *Line-oriented IO*
    - *Formatted IO*
- **Binary streams:**
  - Streams composed of just data.

Opening/closing Files

The `FILE*` type represents an open stream.

The **fopen** function opens a file.

```c
FILE *fopen(char *filename, char *mode);
```

- The arguments to fopen are strings.
- `mode` specifies that access mode. There are several options:
  - **"r"** (read) - the file must already exist.
  - **"w"** (write) - if the file already exists, overwrite it.
  - **"a"** (append) - if the file does not exist, create it.
- Returns NULL if the function fails to open the file.
- fopen's return value is often used in if-statements.

The **fclose** function closes a file and also flushes the output, if needed.

```c
int fclose(FILE *fp);
```

- Returns 0 on success.
- Failure can occur for numerous reasons, such as OS interruptions.

Line-oriented IO

The **fgets** function reads characters from the given stream and stores it in the given buffer.

```c
char *fgets(char *buf, int size, FILE* stream);
```

- The function stops when either:
  - A newline is read.
  - `size - 1` characters are read.
- A null character is always appeneded to strings.
- Unlike gets, fgets includes the newline in `buf`.

> **Note**
>
> Replacing the newline character with the null character may be needed due to printing usually being done with an appended newline character.

- NULL is returned on error or EOF.
- On success, `buf` is returned.

The **fputs** function prints a string to a given stream.

```c
int fputs(const char *str, FILE *stream);
```

- Returns a nonnegative value on success, and returns an EOF otherwise.
- Equivalent functionality to `fprintf(stream, "%s", str)`.

The **feof** function checks if the EOF has been reached for the specified file stream.

```c
int feof(FILE *stream);
```

- Returns 0 if the EOF has been reached.
- Otherwise, a nonnegative integer is returned.

> **Note**
>
> **EOF** is a macro defined in stdio.h that represents the end of a file.

Streams

There are three predefined streams in every program:
- **stdin**: standard input (redirect using <)
- **stdout**: standard output (redirect using >)
- **stderr**: standard error (redirect using >&)

The type of steams is `FILE*`.

We can use the **fprintf** function to direct output to any specified stream.

```c
int fprintf(FILE *stream, char *fmt, ...);
```

- When stdin is passed in as the stream, it is analogous to printf.

Accordingly, we can use **fscanf** to read from a given stream.

```c
int fscanf(FILE *stream, char *fmt, ...);
```

- When stdout is passed in as the stream, it is analogous to scanf.

We can also separately store data from different kinds of streams.
- For instance, we can store the data in stdout and stderr in different text files.

```shell
a.out < predefined_data.txt > results.txt >& errors.txt
```

We can use the shortcut **ctrl + d** to indicate EOF in Linux. While **ctrl + c** terminates, it does not indicate EOF.

`sscanf()`

The **sscanf** function reads formatted input from `str`.

```c
int sscanf(char str[], char* fmt, ...);
```

- Returns the EOF if nothing is read (`fmt` is an empty string/string with only whitespace).
- Otherwise, returns the number of values retrieved.

`sprintf()` and `snprintf()`

The **sprintf** function formats the output and stores it in the provided buffer. Afterward, the null byte is added.

```c
int sprintf(char *buf, char *fmt);
```

The **snprintf** function does the same thing, but only stores up to $n - 1$ characters in the provided buffer before adding the null byte.

```c
int snprintf(char *buf, size_t limit, char *fmt);
```

Reading data from strings

To read data from strings, fgets can be used to read a line. Then, the line may be parsed using sscanf.

```c
#define MAX 1024
char buf[MAX + 1];
int a, b;

while (fgets(buf, MAX + 1, stdin) != NULL) {
  if (sscanf(buf, "%d %d", &a, &b) == 2) {
    printf("%d %d\n", a, b);
    break;
  }
}
```

Working with long lines

> **Note**
>
> Line lengths can vary greatly. Thus, programs either repeatedly read a buffer until they reach the end of a line or enforce a maximum line length.

Shell Programs

Shell programs can be used to interact with the terminal.

- The first line of a shell program, the shebang line, is denoted by #!. This line can be used to determine the interpreter (syntax) to use to run the pogram.

```
#!/bin/bash
```

- Lines other than control structures can represent UNIX commands.

The following is an example of a shell program using C shell:

```
1    #!/bin/csh
2
3  if ($#argv != 1) then
4     echo "Usage: mrm <file/directory>"
5     exit(1)
6  endif
7
8  echo ""$0" will move "$1" to local tmp_junk_dir"
9  mv $1 ~/tmp_junk_dir
10 if ($? != 0) then
11    echo ""
12    echo "Terpito your remove script failed"
13    echo ""
14 else
15    echo ""
16    echo "file/directory moved to ~/tmp_junk_dir"
17    echo ""
18 endif
```

10 Dynamic Memory Allocation

Memory Organization

> **Note**
>
> Refer to lab 6.

While stacks often retain memory, heaps usually delete unused memory. Depending on the system implementation, either the unused memory space or the top of the heap will be used.

Dynamically Allocated Memory

There are many reasons for dynamically allocating memory.

- Eliminate compile-time limits
  - Data structures' sizes are generally unnown until runtime.
  - Allocating a lot of space to store all possible scenarios can either be error-prone or wasteful.
- **Garbage collection:**
  - The programmer does not have to keep track of objects.
  - However, the execution will be slower because the collection is expensive and needs to be ran periodically.
- **User-managed heap:**

Memory Management Functions

The **malloc** function allocates the specified amount of memory, if available from the heap.

```c
void *malloc(size_t amount);
```

- Returns a generic pointer to the beginning of the allocated memory.
  - Usually assigned to an integer pointer.
- Returns NULL if the allocation fails.
- **None of the space is initialized.**

The **calloc** function allocates the amount of memory from the heap analogous to the provided object count and object size.

```c
void *calloc(size_t count, size_t object_size);
```

- Also returns a generic pointer to the beginning of the allocated memory.
- Returns NULL if the allocation fails.
- **The entire space is initialized to 0.**

The **free** function frees the memory pointed at by the specified pointer.

```c
void free(void *ptr);
```

- free does not change the provided pointer, but can change the memory it points to.
  - The provided pointer becomes a **dangling pointer**.
  - By convention, these pointers should be set to NULL because they should not be dereferenced.
- The NULL pointer can be safely freed, although it does nothing.

> **Important**
>
> `ptr` is required to point toward memory previously allocated by either malloc or calloc.

Pointer Aliases

Two pointers can point to the same address. This is known as a **pointer alias**.

Common Errors

> **Caution**
>
> 1) Dereferencing pointers to freed space.
> 2) Not NULL checking malloc's return value.
> 3) Not initializing the memory returned by malloc.
> 4) Attempting to free non-heap memory.
>   a) The specified pointer should be allocated by malloc or calloc.
> 5) Freeing something that is not the beginning of a dynamically allocated block.
> 6) Not allocating enough space.
>
> ```c
> char str[] = "TEST";
> char *p = malloc(strlen(str)); // Needs +1 to account for null byte.
> ```
>
> 7) Using the incorrect size for memory allocation.
> 8) **Memory leaks:**
>   a) Occurs when dynamically-allocated memory is not freed. Losing the only pointer to some dynamically-allocated chunk of memory leads to a memory leak.
>   b) While not immediately fatal, it can waste resources and eventually halt the program.

11 Dynamic Memory Allocation

Linked Lists

Much like Java's objects with references to objects of the same class, C structures can contain pointers to structures of the same type.

- Thus, using structs, we can define linked lists by defining some head node as a **Node pointer**.

```c
typedef struct node {
  int data;
  struct node *next;
} Node;
```

If an anonymous structure is preferred, then a **forward declaration** can be used to inform the compiler about Node.

```c
typedef struct node Node;
typedef struct {
  int data;
  struct node *next;
} Node;
```

In memory, the head node can be declared locally while the remaining nodes are stored on the heap space.

- **Dynamic memory allocation is useful for creating or removing new nodes in functions.**
- For instance, memory must be allocated using malloc or calloc when inserting new nodes in a separate function.
  - Nodes with string fields require another malloc or calloc call to free the necessary space for them.

> **Caution**
>
> The order of the dynamic memory allocation matters. When freeing space, free any string fields before freeing the node.

Operations

> **Note**
>
> Often, passing the address of the head node is needed for operations in the case that the head node is the target.

- Inserting a node.
- Deleting a node.

Dummy Nodes

Dummy nodes contain specific data that makes them distinguishable from other nodes. They can be used to simplify insertions and deletions.

- The head node can point to a dummy node, then operations can start from the dummy node.
- Thus, *the head node's address won't have to be passed as arguments to operation functions.*

Doubly linked Lists

Doubly linked lists are similar, albeit with references to both the previous and next nodes.

```c
typedef struct node {
  int data;
  struct node *prev;
  struct node *next;
} Node;
```

Reallocation

There may be cases where more memory must be manually allocated.

- While allocation could be done by copying the values to some larger allocated memory, the **realloc** function can be used to simplify this process.

Th realloc function reallocates given memory.

```c
void *realloc(void *p, size_t new_size);
```

$$\text{original size} \geq \text{new size}$$

- realloc performs reallocation in place, then returns p.

Operating on blocks of memory

```c
void *memcpy(void *dst, void *src, size_t n);
void *memmove(void *dst, void *src, size_t n);
```

12 Lab 1 and 2

Wildcard and Path symbols

Directories in filepaths are separated by forward slashes (/). Specific characters denote specific directories.

- The slash (/) refers to the root directory.
- The dot (.) refers to the current directory.
- The two dots (..) refer to the parent directory.
- The tilde (~) refers to the home directory.
- The asterisk (*) refers to all files in a directory.
  - This can be used alongside a file extension to represent all files of a specific type.
- The question mark (?) matches one character in a filename.
  - To match numerous characters, multiple question marks can be chained.

Linux Shell Commands

The following commands are the *essentials* for writing and executing C programs.

- **Flags** are extra options for commands. They are generally denoted by a dash.
- The up-arrow and down-arrow keys enable you to navigate through previous-ran commands.
- Filename completion can be done using the tab key.

```shell
pwd
```

1) `pwd` displays the current direction. Forward slashes separate directories and files.
2) `ls` displays all files and directories in the current directory.
   a) `ls -F` will append specific characters to items' names. Particularly, directories are marked with a forward slash.
   b) `ls -l` will display results in the long list format.
   c) `ls -a` will also include hidden directories and files (including current directory and parent directory).
   d) `ls -al` combines the previous two flags.
3) `cd` changes the directory to the home directory.
   a) `cd DIRECTORY_NAME` changes the directory to the specified directory.
   b) `cd ~` makes the current directory the home directory.
4) `mkdir` creates a new directory in the current directory.
5) `rmdir DIRECTORY` removes the specified *empty* directory.
   a) `rmdir -f -r DIRECTORY` removes the specified directory, *even if it is non-empty.*
6) `cp FILE_1 FILE_2` will create a new file using the data of the first file.
   a) `cp -r FILE_1 FILE_2` will create a new file using the data of the first file, but recursively. **This is required when copying folders.**
7) `rm FILE` permanently removes the specified file.
   a) `rm -r DIRECTORY` permanently removes the specified directory.

8) `mv` moves files/directories to a directory or renames files/directories.
   a) `mv FILE_1 FILE_2` renames the first file to `FILE_2`.
   b) `mv FILE DIRECTORY` moves the file into the specified directory.
   c) `mv DIRECTORY_1 DIRECTORY_2` renames the first directory to `DIRECTORY_2` if the second directory doesn't exist. Otherwise, the first directory is moved to the second directory.
9) `touch FILE` updates the access and modification times of FILE to the current time. *If FILE does not exist, then an empty file is created*.
10) `man COMMAND` pulls up the documentation for a given command.
    a) `man SECTION_NUMBER COMMAND` pulls up the specified section for the documentation of the given command.
11) `diff FILE_1 FILE_2` shows the differences between `FILE_1` and `FILE_2`.
    a) `diff -bwi FILE_1 FILE_2` also ignores whitespace differences.
    b) `diff -Bwi FILE_1 FILE_2` also ignores whitespace and newline differences.

13 Lab 4

This lab will focus on debugging tools.

`diff` and I/O redirection

The `diff` command was covered in labs 1 and 2.

- `diff FILE_1 FILE_2`
- As explained prior, it displays differences between two files.
- The `-bwi` flag ignores whitespace that isn't empty lines.
- The `-Bwi` flag ignores whitespace in general.

`diff` works best when used with input/output redirection while running a program.

- The less-than symbol (<) allows input to be read from a specified file.

```
1    a.out < public_test.in
```
Shell

- The greater-than symbol (>) allows the output to be written to a specified file.
  - By default, this will not overwrite an existing file. Using (>!) allows the specified file to be overwritten if it exists.

```
1    a.out > results
2    a.out >! results
```
Shell

Projects and exercises will be bundled with public test files and their corresponding files. Thus, we can do the following to confirm the validity of the tests.

```
1    gcc prog.c
2    a.out < public0?.in >! results
3    diff -U 0 -b results public0?.output
```
Shell

Valgrind

The **valgrind** utility is a tool that provides extra information when compiling a program.

- valgrind can help locate problems in code when there is a segfault.
- valgrind can also detect invalid memory operations.

To use the valgrind utility, compile your program with the `-g` flag, then execute using valgrind.

```
1    valgrind a.out
2    valgrind --track-origins=yes a.out
```
Shell

- The track-origins flag shows where uninitialized values come from.

We can also apply I/O redirection to valgrind.

```
1    valgrind a.out < data.txt > results
```
Shell

gdb

gdb is a C debugger. To use gdb, compile the desired file with the `-g` flag, then execute using `gdb`.

```Shell
gcc -g -o prog.c prog
gdb prog
```

Shortcuts:

- **q** (quit) exits.
- **l** (list) lists all commands.
- **r** (run) runs the program.
- **b** (break) sets a breakpoint.
  - At least one breakpoint is needed to prevent the program from being run to completion.
  - Both line numbers and function names work as locations.
    - `b 7` sets a breakpoint on line 7.
    - `b func` sets a breakpoint at the function `func`.
- **p** (print) inspects variables.
- **n** (next) executes a statement/function call.
- **s** (step) executes a statement/function call steps into the function.
- **c** (continue) continues execution.
- **where** lists the stack frame.
  - After a segfault occurs, this can be used to identify the source.
- **frame** changes the stack frame to a given input.
  - `frame 1`
- **clear** clears a breakpoint
  - `clear 25` clears a breakpoint on line 25.
- **delete** deletes all breakpoints.
- ^x + a opens a graphical interface

Like when running an executable in the terminal, I/O redirection may be used when running a program in gdb.

```gdb
r < public01.in > results
```

splint

splint is a tool for statically checking C programs.

- To use it, run the splint command on a C program.

```Shell
splint prog.c
```

assert

The assert tool is a macro that prints an error message and terminates the program if the provided expression is false. It is defined in the header file **assert.h**.

```c
#include <assert.h>

int x = 0;

assert(x); //terminates program
```

There is a compiler option for disabling assert:

```shell
gcc -D NDEBUG prog.c
```

Compiler Options

There are many compiler flags we can use with `gcc` for debugging.

- The fsanitize flag can help identify memory problems such as buffer overflow.

```shell
gcc prog.c -fsanitize=address
```

14 Lab 5

`emacs`

`emacs` is a text editing utility in the linux shell.

The following shortcuts are important.

- **^x + ^c** quits.
- **^x + ^s** saves a file.
- **^x + ^f** opens a file.
- **^k** kills the process.
- **^y** yanks (pastes) previously killed text.
- **^s** forward searchs.
- **^r** reverse searchs.
- **^_** undos last change.
- **^x + ^b** lists all buffers.
- **^x + k** kills the current buffer.
- **^x + 1** shows the current buffer.
- **^x + 2** shows two buffers side by side.
- **^g** cancels the current command.

15 Lab 6

Comma Operator

The comma (,) is an operator with two operands.

- It evaluates the left operand, then the right operand.
- Commas have the lowest precedence of all operators.

```c
int i;
i = 1, 2, 3, 4; //i = 1, assignment is an operator with high precedence
i = (1, 2, 3, 4) //i = 4, parentheses have very high precedence, and commas evaluate left -> right
```

C Memory Organization

Memory in C from top to bottom is organized as follows:

- **Stack**
  - Local variables and function parameters
  - Grows downward as a function is called, then shrinks upwards as the function returns.
  - Variables disappear when a function returns.
- **Heap**
  - Dynamically allocated memory
  - Grows upward, but generally does not shrink.
  - Variables last until they are deallocated.
- **Data**
  - Globals and statics
- **Text**
  - Compiled code and constants

> **Note**
>
> The organization here is the binary format for grace. Some systems have different priorities for data and text.

Identifier Scope

There are two main types of scope in C.

- **Block:** variables declared inside blocks is only visible within the block and nested blocks within.
- **File:** identifiers declared outside of any block is visible everywhere in the file *after declaration*.
  - **Global variables** are variables defined in this scope. By default, they are assigned to 0. *Symbolic constants should be used over global variables*.

> **Important**
>
> Scope applies to both global variables and function names.

Storage

There are two types of storage.

- **Automatic:** Temporarily allocated storage that expires after the end of the block/ function call.
  - By default, all block-scoped variables use automatic storage.
- **Static:** Exists for the entire life of the program.
  - By default, global variables use static storage.
  - Initializations to static variables only occur once.

Block-scoped variables can be changed to use static storage.

Identifier Linkage

Encountering two instances of the same identifier across different files is a common issue.

- **Linkage** is the property of an identifier that determines if multiple declarations of the same identifier refer to the same entity.

There are three types of linkage:

- **None:** All declarations of an identifier refer to different entities. *A variable with no linkage belongs to a single function and cannot be shared*.
- **Internal:** All declarations of an identifier inside a given file refer to the same entity, but declarations across files refer to different entities.
  - The modifier **static** is used to specify internal linkage.
    - If not initialized, a static variable is initialized to 0.
- **External:** All declarations of an identifier refer to the same entity.
  - By default, all global variables and functions have external linkage.
  - The modifier **extern** is used to specify external linkage.

Enumerated Types

Enumerated types can be used to represent things that only take on certain values. For example, we can represent card suits using enums.

- Denoted using `enum`.

By default, the values within an enum are equal to $0, 1, 2\ldots$ based on the order of definition

- However, values can be specified as other integers.

```c
#include <stdio.h>
int main() {
  enum Suit {
```

```
4         SPADES, HEARTS,
5         DIAMONDS = 42, CLUBS
6       };
7       enum Suit suit1, suit2;
8       suit1 = SPADES;
9       suit2 = CLUBS;
10      if (suit1 < suit2) {
11        printf("Spades are first.\n");
12      } else {
13        printf("Clubs are first.\n");
14      }
15
16      printf("Spades = %d, Clubs = %d\n",
17      suit1, suit2);
18
19      return 0;
20    }
```

Lvalues and Rvalues

An **rvalue** is anything that can appear on the right side of an assignment statement.
- Generally any expression.

An **lvalue** is anything that can appear on the left side of an assignment statement.
- Values that represent a location to store a value, such as variables or array entries.

Thus, different sides of an assignment statement are treated differently.

Typecasting

When two operands of different types are involved in an expression, the lower rank operand will be converted to the type of the higher rank operand.
- For instance, given an expression with a double and float, the float would be converted to a double.
- This is **implicit typecasting.**

The following types are ranked from highest to lowest. rank.
- long double
- double
- float
- unsigned long int
- long int
- unsigned int
- int
- char, short

Implicit type casting also occurs during assignment, however this can often lead to precision loss.

> **Note**
>
> The compiler may warn you of precision loss due to implicit casting.

16 Lab 8

Standard Library (continued)

getchar is defined as follows:

```c
int getchar(void);
```

- Returns the first character in the input stream as an int (ASCII value).
- Returns EOF when the end of the input stream is reached or there is an input error.
- getchar is commonly used to flush out extra newlines or whitespace in the input stream:

```c
scanf("%ls", str);
(void) getchar(); //Casting to void discards the resulting expression
```

putchar is defined as follows:

```c
int putchar(int c);
```

- Writes the specified character to the program's output.
- Returns the character's ASCII value as an int.

ungetc is defined as follows:

```c
int ungetc(int c, FILE *stream);
```

drgioaijgiahgauhgaugwri

String Library (continued)

strcat is defined as follows:

```c
void strcat(char *dest, const char *src);
```

- Concatenates the string in `src` to the end of `dest`.

> **Caution**
>
> Like with strcopy, strcat does not know if `dest` has enough space to support concatenation.

strstr is defined as follows:

```c
char *strstr(char *str, char *substring);
```

- Returns a char pointer to the first instance of `substring` in `str`.
- If no instance can be found, the NULL pointer is returned.

strncmp is defined as follows:

```c
int strncmp(const char *s1, const char *s2, size_t n);
```

- Does the same thing as strcmp, but only for the first n characters.

strncpy is defined as follows:

```c
void strncopy(char *dest, const char *src, size_t n);
```

- Does the same thing as strcpy, but only for the first n characters of `src`.

> **Caution**
>
> Consequently, strncpy does not always add the null byte. Thus, manual addition to the destination string is often necessary.

Command Line Arguments (continued)

Command line functionality is built into the main function using the parameters `int argc` (argument count) and `char **argv` (argument values).

If we compile some C program to the default executable, the values of the command line parameters may be as follows:

```c
int argc = 2;
char **argv = {'.a/out', 'field1'};
```

2-dimensional Arrays (continued)

Similar to 1-dimensional arrays, having an incomplete initializer will result in the unprovided values defaulting to 0.

- This is because they are organized similarly to 1-dimensional arrays in memory.

```c
#define MAX_ROWS 2
#define MAX_COLS 3

int arr1[MAX_ROWS][MAX_COLS] = {{1, 2, 3}, {4, 5, 6}}
// {{1, 2, 3}, {4, 5, 6}}
int arr2[MAX_ROWS][MAX_COLS] = {{1}, {3}} // {{1, 0, 0}, {3, 0, 0}}
int arr3[MAX_ROWS][MAX_COLS] = {1, 4, 9, 16, 25, 36}
// {{1, 4, 9}, {16, 25, 36}}
int arr4[MAX_ROWS][MAX_COLS] = {5} // {{5, 0, 0}, {0, 0, 0}}
int arr5[MAX_ROWS][MAX_COLS] = {{6}, 2, 9} // {{6, 0, 0}, {2, 9, 0}}
```

17 LAB

File Parsing

When parsing files using **feof** as a while condition, unintended behaviors may occur when parsing the end of some files.

```c
while (!feof(input)) {
  fgets(line, MAX + 1, input);
  fprintf(stdout, "%s", line);
}
```

- If there is no newline before the EOF, the output will not end with a newline.
- If there is a newline before the EOF, the last line will be printed twice because the

Now, if the order of the statements is swapped, the last line will not be printed

Preprocessing

Preprocessing is the first stage of compilation.

- The compiler flag -E stops the compilation at the preprocessing stage.

There are several preprocessor predefined symbols:

- **__FILE__** - Represents a string containing the filename where the symbol is being used.
- **__LINE__** - Represents a string containing the line number where the symbol is being used.
- **__DATE__** - Represents a string containing the current date in the system's timezone using the format **Mmm _d yyyy**.
- **__TIME__** - Represents a string containing the current time in the system's timezone using the format **hh:mm:ss**.

Similarly, the **#define** directive can be used to create macros denoted by the specified symbolic constant.

Conditional Compilation

We can compile programs conditionally to account for different environment.

#if, **#elif**, and **#else** are all analogous to their counterparts. **#endif** is used to end a conditional block.

There are also **preprocessor operators.** Namely, **defined()** is often used in condition compilation to only define something if it has not bene previously defined:

```c
#if !defined(MAX_STR_SIZE)
#define MAX_STR_SIZE 80
#endif
```

Symbols may also be defined at compile time by using their names as flags:

```shell
gcc -D MAX_STR_SIZE=100 prog.c
```

Given that MAX_STR_SIZE is a symbolic constant in prog.c

File Inclusion

Additional files may be included using the **#define** directive.

- Because inclusions essentially import definitions into the program, including a program that includes another can lead to multi-definitions (error).
- Thus, we can use conditional compilation in files to ensure the safety of their use:

```c
#if !defined(COORDS)
#define COORDS

typedef struct {
  int x, y, z;
} Coords;

#endif
```